

Modern Storages and Data Warehousing Week 6 - DWH

Попов Илья, i.popov@hse.ru

Ресар прошлых занятий

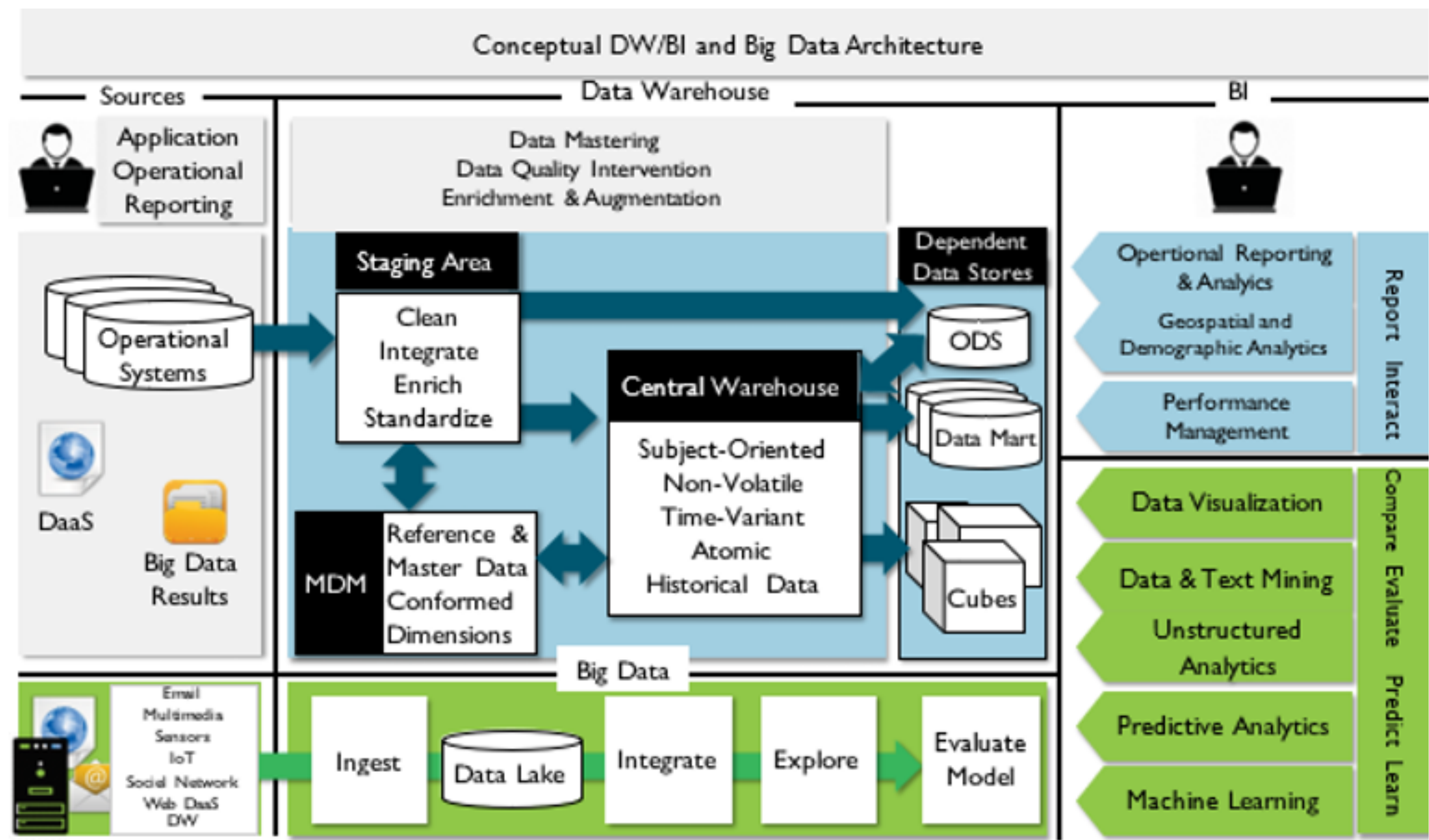
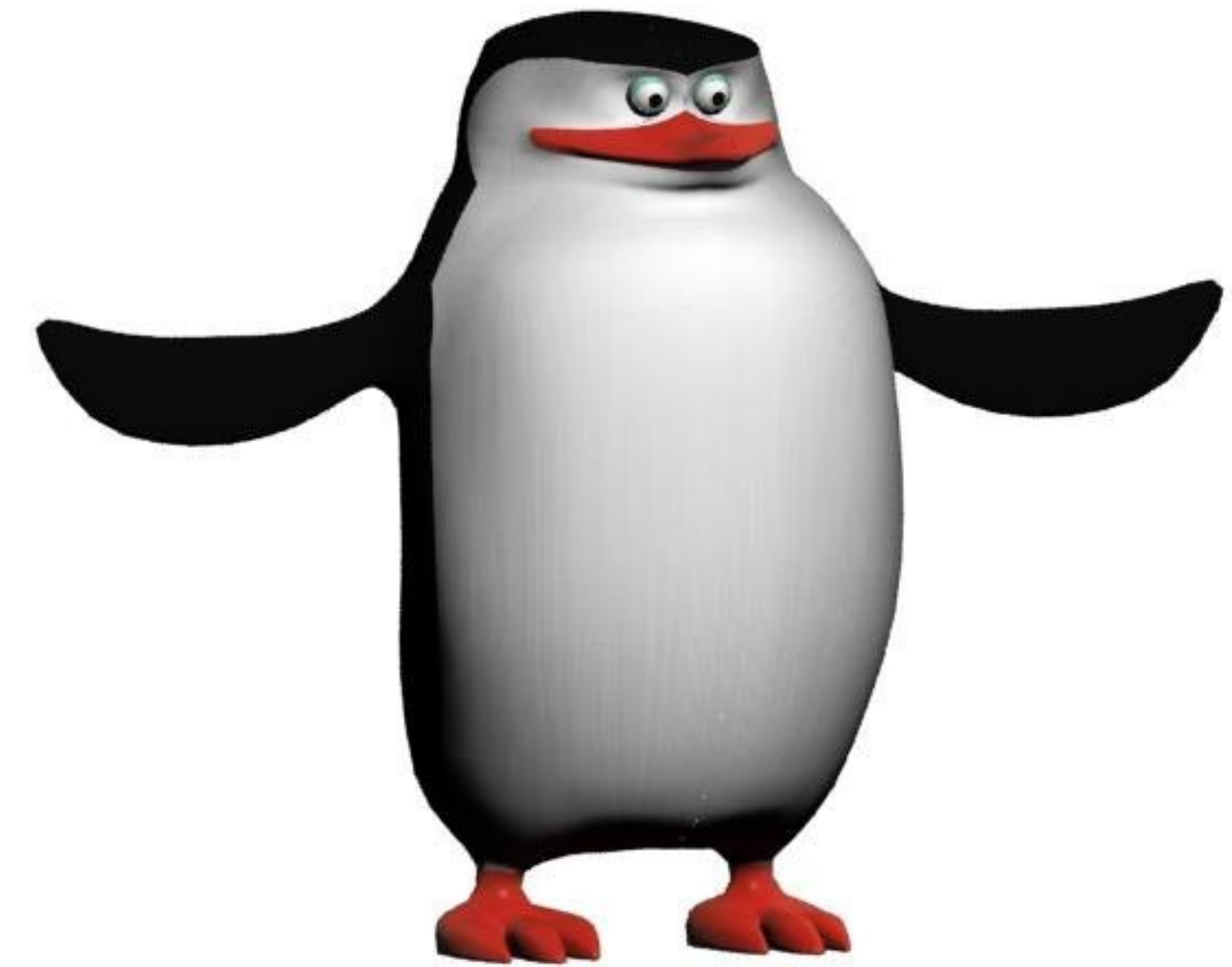


Figure 5: Data Warehouse Concept

1- DWH. Теормин

Первичный ключ

Был курс по базам?
Помним, что такое PRIMARY KEY?



пингвин для привлечения внимания

Первичный ключ

Будем называть первичным ключом подмножество атрибутов отношения, если оно соответствует условиям:

- › **Уникальность** - в отношении не может быть более одного кортежа с одинаковым значением
- › **Неизбыточность** - никакое его подмножество не обладает свойством уникальности

Потенциальный ключ из одного атрибута называется **простым**, из нескольких атрибутов - **составным**.

Отношение может иметь несколько потенциальных ключей. Тогда среди них выделяются первичный и альтернативные.

Нормальные формы

**Был курс по базам?
Помним, что такое нормальные
формы?**



котик для привлечения внимания

Нормальные формы

Здесь и далее - примеры из моей любимой статьи на Хабре

Отношение находится в **1НФ**, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения. Не должно быть повторений строк в таблице.

Нормальные формы

Здесь и далее - примеры из моей любимой статьи на Хабре

Отношение находится в **1НФ**, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения. Не должно быть повторений строк в таблице.

Вопрос: ниже 1НФ?

Фирма	Модели
BMW	M5, X5M, M1
Nissan	GT-R

1НФ

Нарушение нормализации 1НФ происходит в моделях BMW, т.к. в одной ячейке содержится список из 3 элементов: M5, X5M, M1, т.е. он не является атомарным.

Настоящая 1НФ:

Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

2НФ

Отношение находится во 2НФ, если оно находится в 1НФ и каждый не ключевой атрибут неприводимо зависит от первичного ключа.

2НФ

Отношение находится во 2НФ, если оно находится в 1НФ и каждый не ключевой атрибут неприводимо зависит от первичного ключа.

Вопрос: ниже 2НФ?

<u>Модель</u>	<u>Фирма</u>	Цена	Скидка
M5	BMW	5500000	5%
X5M	BMW	6000000	5%
M1	BMW	2500000	5%
GT-R	Nissan	5000000	10%

2НФ

Цена машины зависит от модели и фирмы.

Скидка зависят от фирмы, то есть зависимость от первичного ключа неполная.

Исправляется это путем декомпозиции на два отношения, в которых не ключевые атрибуты зависят от ПК.

<u>Модель</u>	<u>Фирма</u>	Цена
M5	BMW	5500000
X5M	BMW	6000000
M1	BMW	2500000
GT-R	Nissan	5000000

<u>Фирма</u>	Скидка
BMW	5%
Nissan	10%

3НФ

Отношение находится в 3НФ, когда находится во 2НФ и каждый не ключевой атрибут нетранзитивно зависит от первичного ключа. Проще говоря, второе правило требует выносить все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы в отдельные таблицы.

Вопрос: ниже 3НФ?

<u>Модель</u>	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

3НФ

Таблица находится во 2НФ, но не в 3НФ.

В отношении атрибут «Модель» является первичным ключом. Личных телефонов у автомобилей нет, и телефон зависит исключительно от магазина.

В результате разделения исходного отношения получаются два отношения, находящиеся в 3НФ:

<u>Магазин</u>	Телефон
Риал-авто	87-33-98
Некст-Авто	94-54-12

<u>Модель</u>	Магазин
BMW	Риал-авто
Audi	Риал-авто
Nissan	Некст-Авто

4НФ, 5НФ и 6НФ

- › **4НФ**: 3НФ + все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей
- › **5НФ**: 4НФ + отсутствуют сложные зависимые соединения между атрибутами
- › **6НФ**: 5НФ + не может быть далее декомпозирована без потерь

Слабая vs сильная нормализация

Критерий	Слабая нормализация (1НФ, 2НФ)	Сильная нормализация (3НФ+)
Адекватность базы данных предметной области	Хуже (-)	Лучше (+)
Легкость разработки и поддержки базы данных	Сложнее (-)	Легче (+)
Скорость вставки, обновления, удаления	Медленнее (-)	Быстрее (+)
Скорость выборки данных	Быстрее (+)	Медленнее (-)

2 - Dimensional Modeling

Философия

- › **Сущность** - это предмет, который существует независимо от других предметов
- › **Атрибут** - это средство, с помощью которого определяются свойства сущности или её связи
- › **Связь** - это отношение между экземплярами двух (или более) разных сущностей

Философия

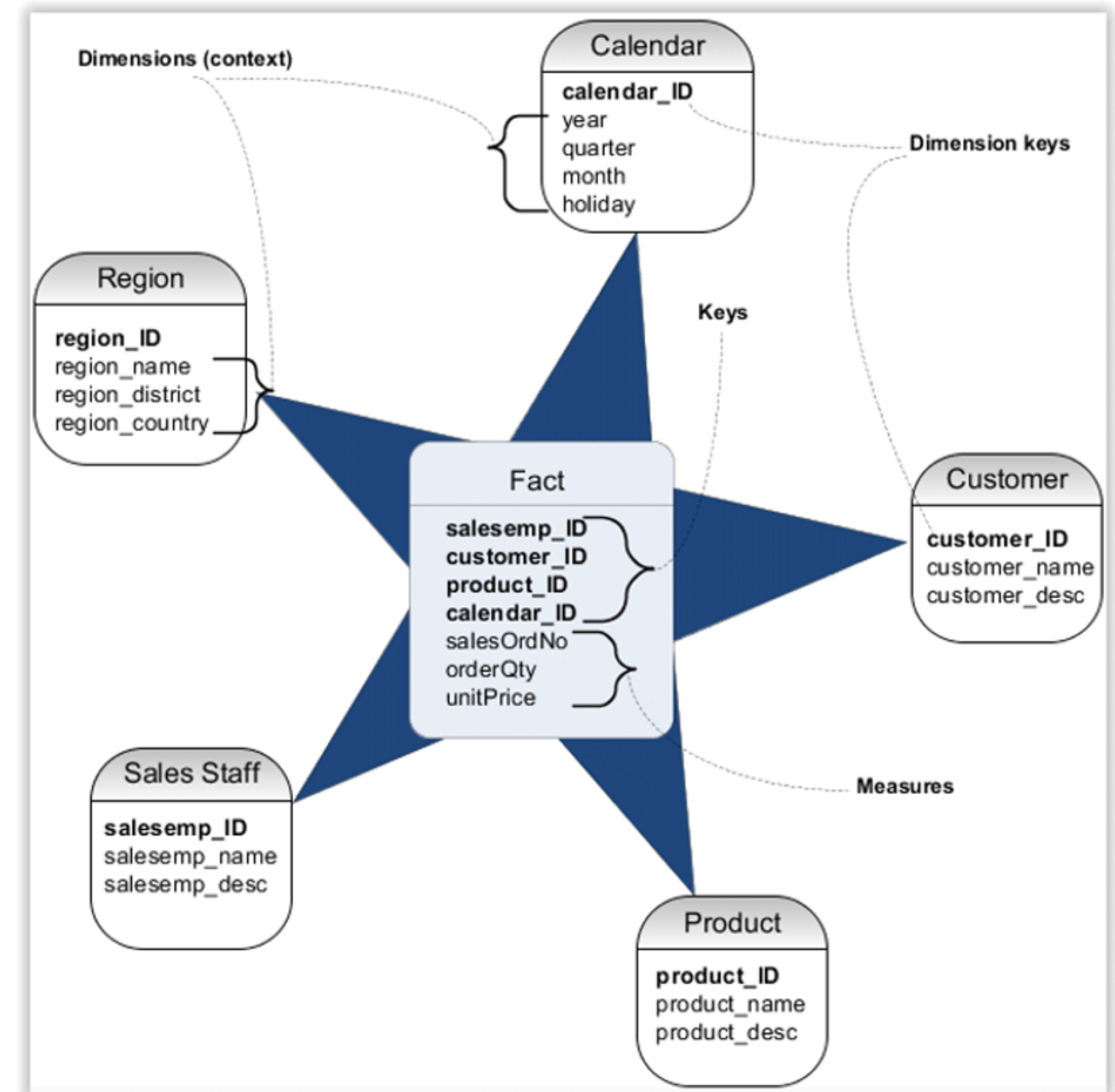
- › **Сущность** - это предмет, который существует независимо от других предметов
- › **Атрибут** - это средство, с помощью которого определяются свойства сущности или её связи
- › **Связь** - это отношение между экземплярами двух (или более) разных сущностей
- › Вопрос: какие виды связей вы знаете?

Философия

- › **Сущность** - это предмет, который существует независимо от других предметов
- › **Атрибут** - это средство, с помощью которого определяются свойства сущности или её связи
- › **Связь** - это отношение между экземплярами двух (или более) разных сущностей
- › Вопрос: какие виды связей вы знаете?
 - Ноль или один**
 - Только один**
 - Ноль или много**
 - Много**
 - Один или много**

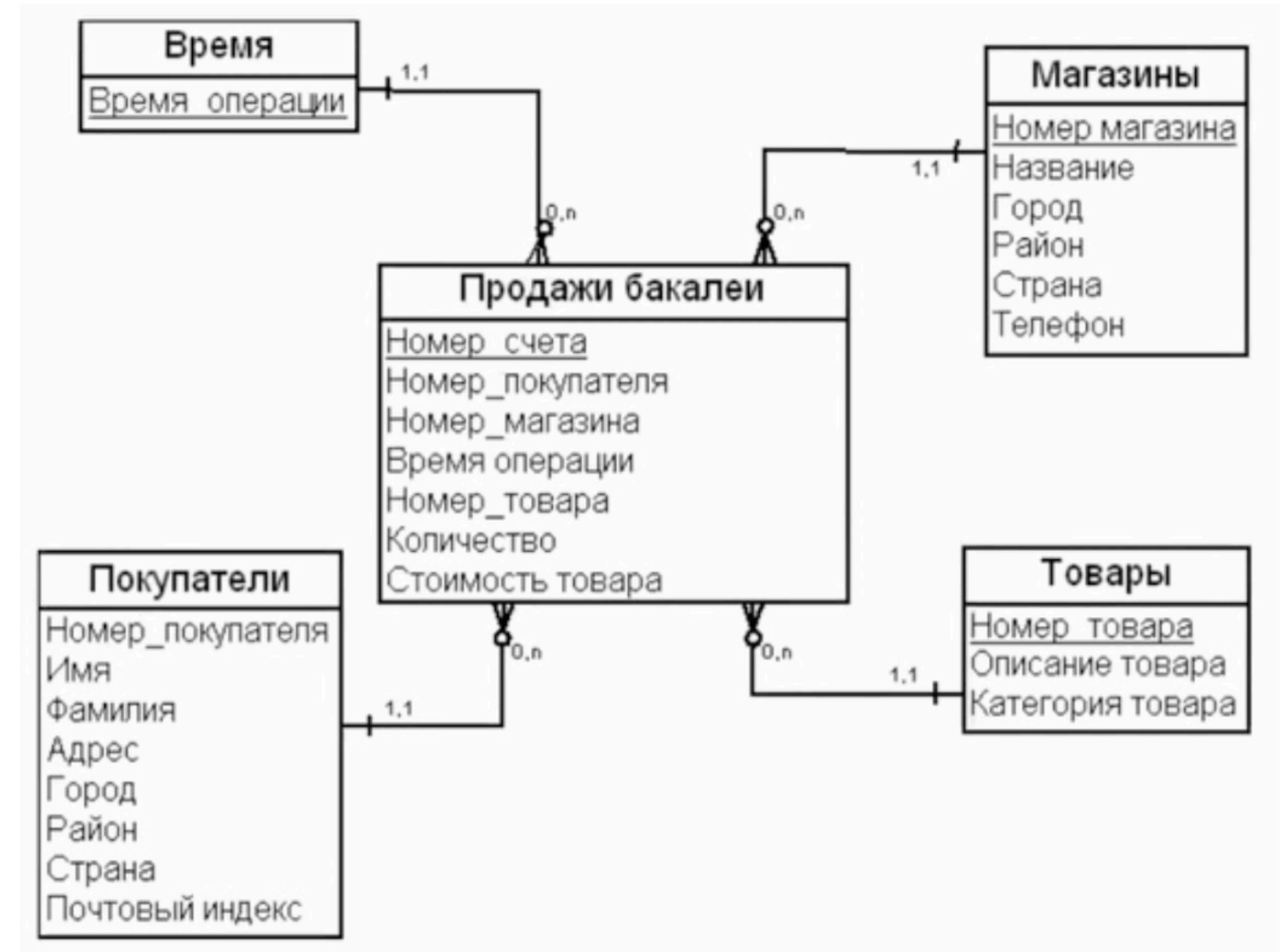
Модель звезда

- › Она же пространственная модель, модель фактов и измерений, модель “сущность-связь”
- › Придумал Ральф Кимбалл
- › Состоит из 2 видов таблиц - **таблиц измерений** и **таблиц фактов**



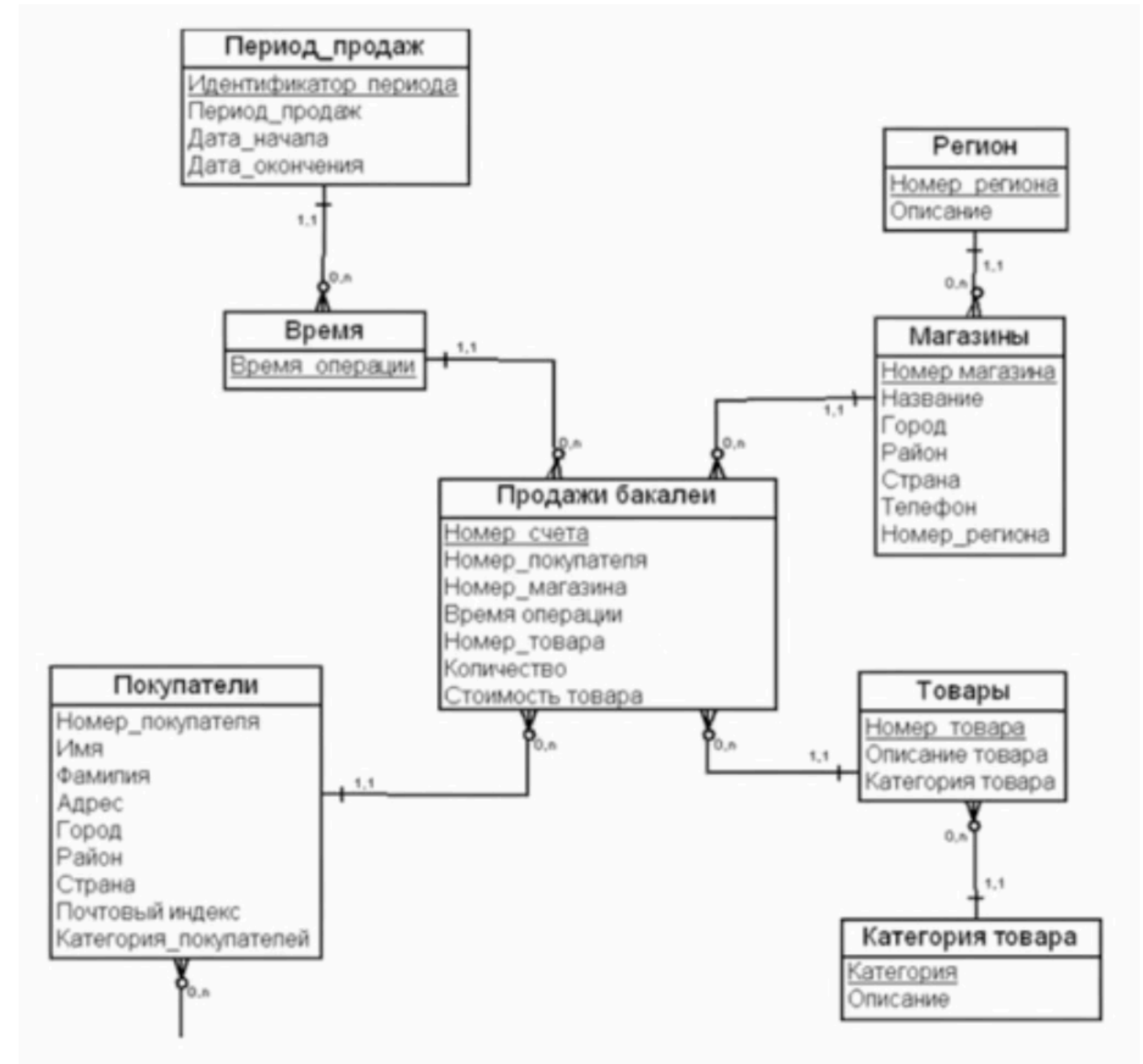
Модель “Звезда”

- › Она же пространственная модель, модель фактов и измерений, модель “сущность-связь”
- › Придумал Ральф Кимбалл
- › Состоит из 2 видов таблиц - **таблиц измерений** и **таблиц фактов**



Модель “Снежинка”

- › Добавляет в модель “Звезда” дополнительные уровни иерархии
- › Измерение “Регион” группирует магазины по регионам
- › Измерение “Категория товара” группирует товары по категориям
- › Измерение “Категория покупателей” группирует покупателей по категориям
- › Измерение “Период продаж” группирует продажи по временным периодам



Модель “Созвездие”

› Несколько таблиц фактов связи между собой какой-то логикой через атрибуты

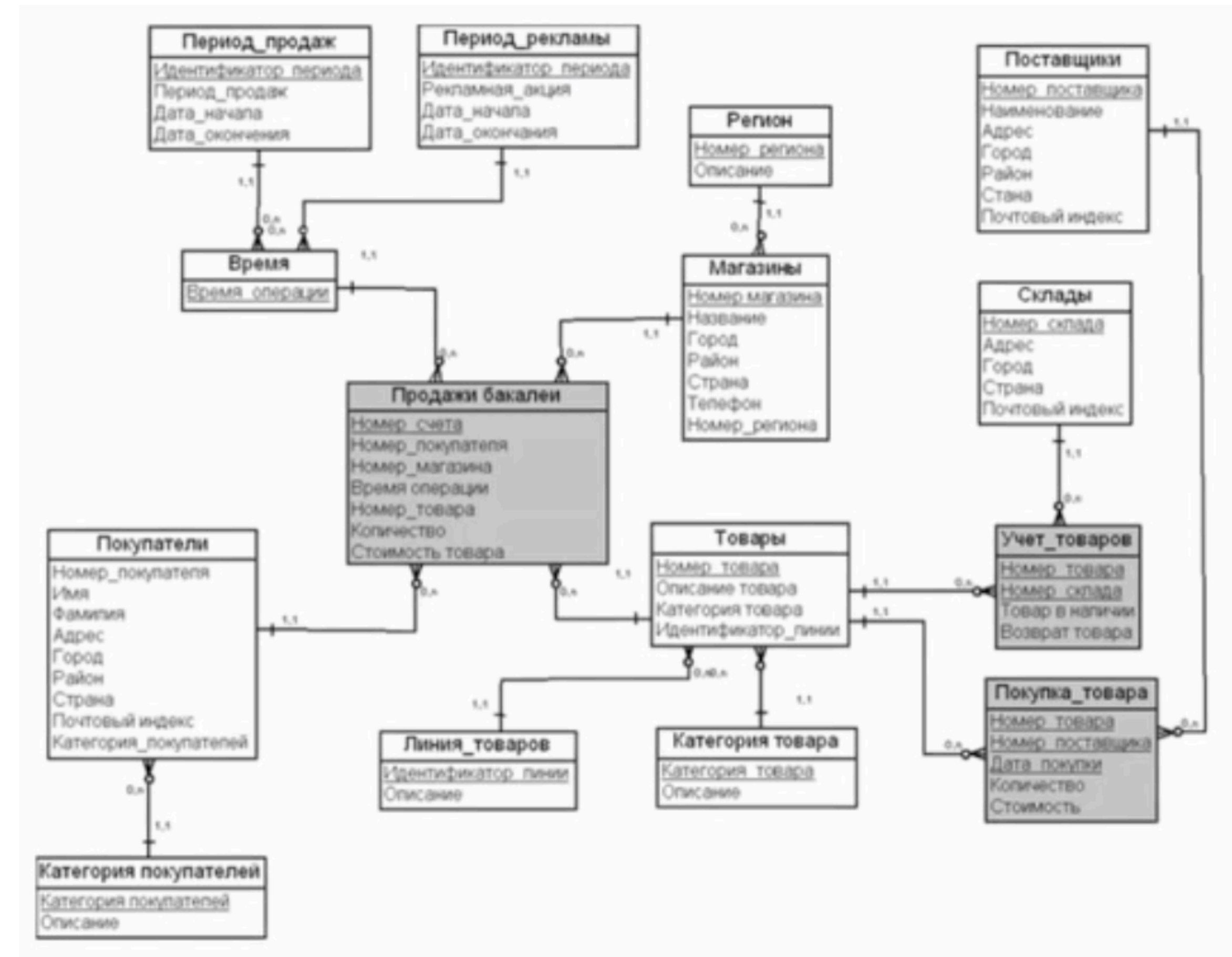


Таблица фактов

Таблица фактов — является основной таблицей хранилища данных. Как правило, она содержит сведения об объектах, событиях или процессах, совокупность которых будет в дальнейшем анализироваться.

Характеристики таблиц фактов:

- › Таблица фактов содержит числовые параметры (метрики),
- › Каждая таблица фактов имеет составной ключ, состоящий из первичных ключей таблиц измерений.
- › Первичный ключ таблицы измерений является внешним ключом в таблице фактов.

Виды фактов

Аддитивные факты (Additive facts). Факт называется аддитивным, если его имеет смысл использовать с любыми измерениями для выполнения операций суммирования с целью получения какого-либо значимого результата. Например, количество продаж, объем продаж и т.д.

Полуаддитивные факты (Semiadditive facts). Факт называется полуаддитивным, если его имеет смысл использовать совместно с некоторыми измерениями для выполнения операций суммирования с целью получения какого-либо значимого результата. Например, остаток на счете, уровень запасов на складе и т.д.;

Виды фактов

- Неаддитивные факты (Non-additive facts).** Факт называется неаддитивным, если его не имеет смысла использовать совместно с каким-либо измерением для выполнения операций суммирования с целью получения какого-либо значимого результата. Например, измерение комнатной температуры.
- Числовые меры интенсивности (Numerical Measures of Intensity).** Факт называется числовой мерой интенсивности, если он, являясь неаддитивным по времени, допускает агрегацию и суммирование по некоторому числу временных периодов. Например, остаток на счете.

Вопросы

Какие виды фактов есть в таблице ниже:

По измерению "Время":					
Дата	Товар	Магазин	Количество продаж	Количество покупателей	Суммарная прибыль
23.01.2009	CD диск	Компьютер	10	10	1500
24.01.2009	CD диск	Компьютер	35	30	5250
25.01.2009	CD диск	Компьютер	20	15	3000
			65	55	9750
По измерению "Товар":					
Дата	Товар	Магазин	Количество продаж	Количество покупателей	Суммарная прибыль
23.01.2009	CD диск	Компьютер	10	6	1500
23.01.2009	Принтер	Компьютер	1	1	5000
23.01.2009	Сканер	Компьютер	2	2	3000
			13		9500
По измерению "Магазин":					
Дата	Товар	Магазин	Количество продаж	Количество покупателей	Суммарная прибыль
23.01.2009	CD диск	Компьютер	10	10	1500
23.01.2009	CD диск	Принтеры	10	5	1500
23.01.2009	CD диск	Оргтехника	20	7	3000
			40	22	6000

Вопросы

Какие виды фактов есть в таблице ниже:

Суммирование метрики "Количество покупателей" по измерению "Товары":					
Дата	Товар	Магазин	Количество продаж	Количество покупателей	Суммарная прибыль
23.01.2009	Бумага для факсов	Компьютер	10	6	1000
23.01.2009	Бумага для принтера	Компьютер	12	7	1320
				13	

Свойства таблиц фактов

Свойства данных в таблицах фактов:

- › Числовые параметры используются для агрегации и суммирования;
- › Значения данных должны обладать свойствами аддитивности или полуаддитивности и по отношению к измерениям, для того чтобы их можно было суммировать;
- › Все данные таблицы фактов должны быть однозначно идентифицированы через ключи таблиц измерений, чтобы обеспечить доступ к ним через таблицы измерений;

Таким образом, таблицу фактов можно разделить на две части. Первая часть состоит из первичных ключей измерений, вторая — из числовых параметров функционально зависящих от ключей таблиц измерений.

Виды таблиц фактов

› Транзакционная таблица фактов (Transaction facts)

В такой таблице фактов сохраняют факты, которые фиксируют определенные события (транзакции). Это факты, описывающие каждое событие бизнеса. Например, продажи товара

› Таблица фактов периодических моментальных снимков (Snapshot)

В такой таблице собирают факты, фиксирующие текущее состояние определенного направления бизнеса. Это факты, которые описывают текущее состояние определенного направления бизнеса для любой комбинации значений измерений за данный период времени. Например, продажи организации на определенную дату (ежедневно)

› Таблица фактов кумулятивных моментальных снимков (Accumulated Snapshot)

В такой таблице собирают факты, фиксирующие некоторое итоговое состояние определенного направления бизнеса на текущий момент времени. Это факты, которые описывают промежуточные итоги деятельности организации по определенному направлению бизнеса для любой комбинации значений измерений за данный период времени. Например, продажи этого года на определенную дату

Виды таблиц фактов

	Транзакционная таблица фактов	Таблица фактов периодических моментальных снимков	Таблица фактов кумулятивных моментальных снимков
Определение гранулярности таблицы фактов	Одна строка на одну операцию	Одна строка на период	Одна строка для периода завершенного события
Измерения	Факты на самом низком уровне детализации “дата/ время”	Факты на конец периода измерения “дата/время”	Факты по нескольким измерениям “дата/время” для фиксации результатов в контрольных точках
Факты	Факты связаны с операционной деятельностью	Факты связаны с периодической деятельностью	Факты связаны с деятельностью, которая имеет определенное время существования
Обновления	Не допускаются	Не допускаются	Допускаются
Кардинальность таблицы фактов	Растет быстро	Растет медленнее, чем в транзакционных таблицах	Растет быстрее, чем в таблицах фактов периодических снимков

Таблицы измерений

Таблица измерений (англ. dimension table) — таблица в структуре многомерной базы данных, которая содержит атрибуты событий, сохраненных в таблице фактов. Атрибуты представляют собой текстовые или иные описания, логически объединенные в одно целое. Таблица измерения имеет первичный ключ и атрибуты, описывающие факты с точки зрения некоторого направления деятельности организации.

Характеристики измерений:

- › Таблицы измерений содержат данные о детализации фактов;
- › Таблицы измерений содержат описательную информацию о числовых значениях в таблице фактов, т.е. они содержат атрибуты фактов;
- › Как правило, таблицы измерений содержат большое количество полей;
- › Таблицы измерений содержат обычно значительно меньше строк, чем таблицы фактов;
- › Атрибуты таблиц измерений обычно используются при визуализации данных в отчетах и запросах;

Виды измерений

- › **Медленно меняющимися измерениями (Slowly Changing Dimensions)** называются таблицы измерений, в которых некоторые атрибуты могут изменить свои значения по истечении некоторого периода времени, причем частота таких изменений является небольшой.
Всего существует 8 основных типов SCD, которые определяют, как история изменений может быть отражена в модели.
- › **Быстро меняющимися измерениями (Rapidly Changing Dimensions)** называются таблицы измерений, в которых некоторые атрибуты могут часто менять свои значения в короткий период времени.

Виды SCD

SCD Type	Dimension Table Action	Impact on Fact Analysis
Type 0	No change to attribute value	Facts associated with attribute's original value
Type 1	Overwrite attribute value	Facts associated with attribute's current value
Type 2	Add new dimension row for profile with new attribute value	Facts associated with attribute value in effect when fact occurred
Type 3	Add new column to preserve attribute's current and prior values	Facts associated with both current and prior attribute alternative values
Type 4	Add mini-dimension table containing rapidly changing attributes	Facts associated with rapidly changing attributes in effect when fact occurred
Type 5	Add type 4 mini-dimension, along with overwritten type 1 mini-dimension key in base dimension	Facts associated with rapidly changing attributes in effect when fact occurred, plus current rapidly changing attribute values
Type 6	Add type 1 overwritten attributes to type 2 dimension row, and overwrite all prior dimension rows	Facts associated with attribute value in effect when fact occurred, plus current values
Type 7	Add type 2 dimension row with new attribute value, plus view limited to current rows and/or attribute values	Facts associated with attribute value in effect when fact occurred, plus current values

Вышло из моды?



3 - Data Vault

Data Vault

Data Vault — набор уникально связанных нормализованных таблиц, содержащих детальные данные, отслеживающих историю изменений и предназначенных для поддержки одной или нескольких функциональных областей бизнеса.

Автор: **Дэн Линстедт** (Dan E. Linstedt)

Дизайн Data Vault — гибок, масштабируем, последователен и приспособляем к потребностям предприятия. Это — модель данных, спроектированная специально для удовлетворения потребностей хранилищ данных предприятия.

В модели Data Vault содержатся структуры, которые знакомые и соответствуют традиционным определениям схемы звезда и 3НФ, включая: измерения, связи многие ко многим и стандартные табличные структуры. Различия заключаются в представлении отношений, структурировании полей и хранении детальных данных, основанных на времени.

Data Vault

Для того чтобы сохранить дизайн простым и изящным, используется минимальное количество типов сущностей: **Хаб** (Hub), **Связь** (Link) и **Сателлит** (Satellite).

Дизайн Data Vault сосредоточен вокруг функциональных областей бизнеса:

- › **Хаб** (Hub) хранит сущности
- › **Связь** (Link) обеспечивает транзакционную интеграцию между Хабами (связи между сущностями)
- › **Сателлит** (Satellite) предоставляет контекст первичного ключа Хаба (атрибуты, описания)

Каждая сущность предназначена для обеспечения максимальной гибкости и масштабируемости, сохраняя при этом большинство традиционных навыков моделирования данных.

Hub

Хабы (Hub) являются отдельными таблицами, содержащими как минимум уникальный список бизнес ключей.

Атрибуты Хаба включают:

- › Ключ бизнес-сущности из внешней системы
- › Суррогатный ключ (Surrogate Key)
- › Временная отметка даты загрузки (Load Date Time Stamp), регистрирующая, когда ключ впервые был загружен в хранилище
- › Источник данных (Record Source) - регистрация исходной системы, используется для обратной трассировки (отслеживания) данных

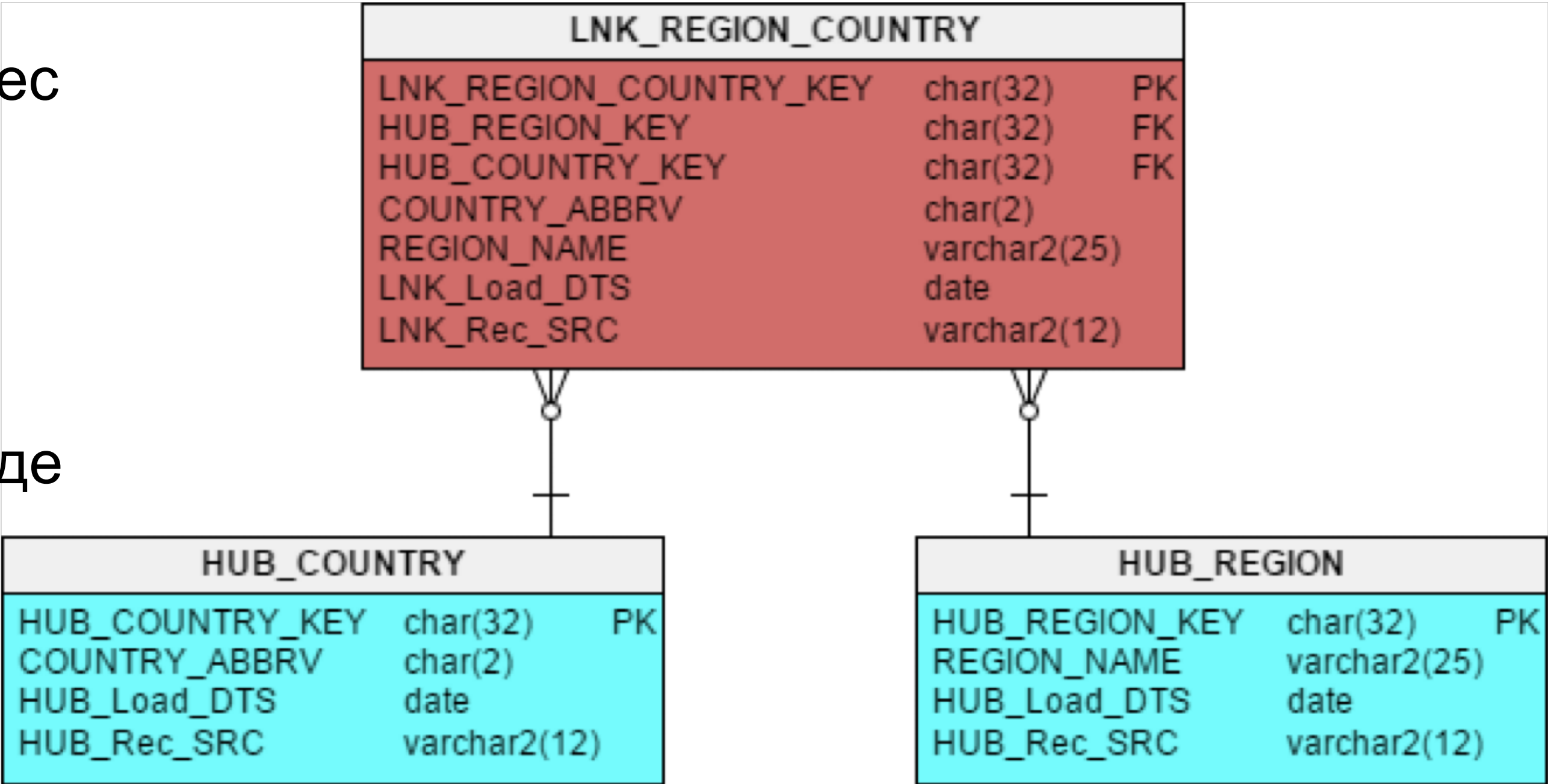
HUB_LOCATION			
HUB_LOCATION_KEY	char(32)	PK	
LOCATION_NAME	varchar2(50)		
HUB_Load_DTS	date		
HUB_Rec_SRC	varchar2(12)		

Link

Связи (Link) представляют собой отношения или транзакцию между двумя или более компонентами бизнеса (два или более бизнес ключа).

Атрибуты линка включают:

- › Суррогатный ключ (Surrogate Key)
- › Ключи Хабов: от 2-ух Хабов до N Хабов, где N – количество входящих в Link Хабов
- › Временная отметка даты загрузки
- › Код источника данных

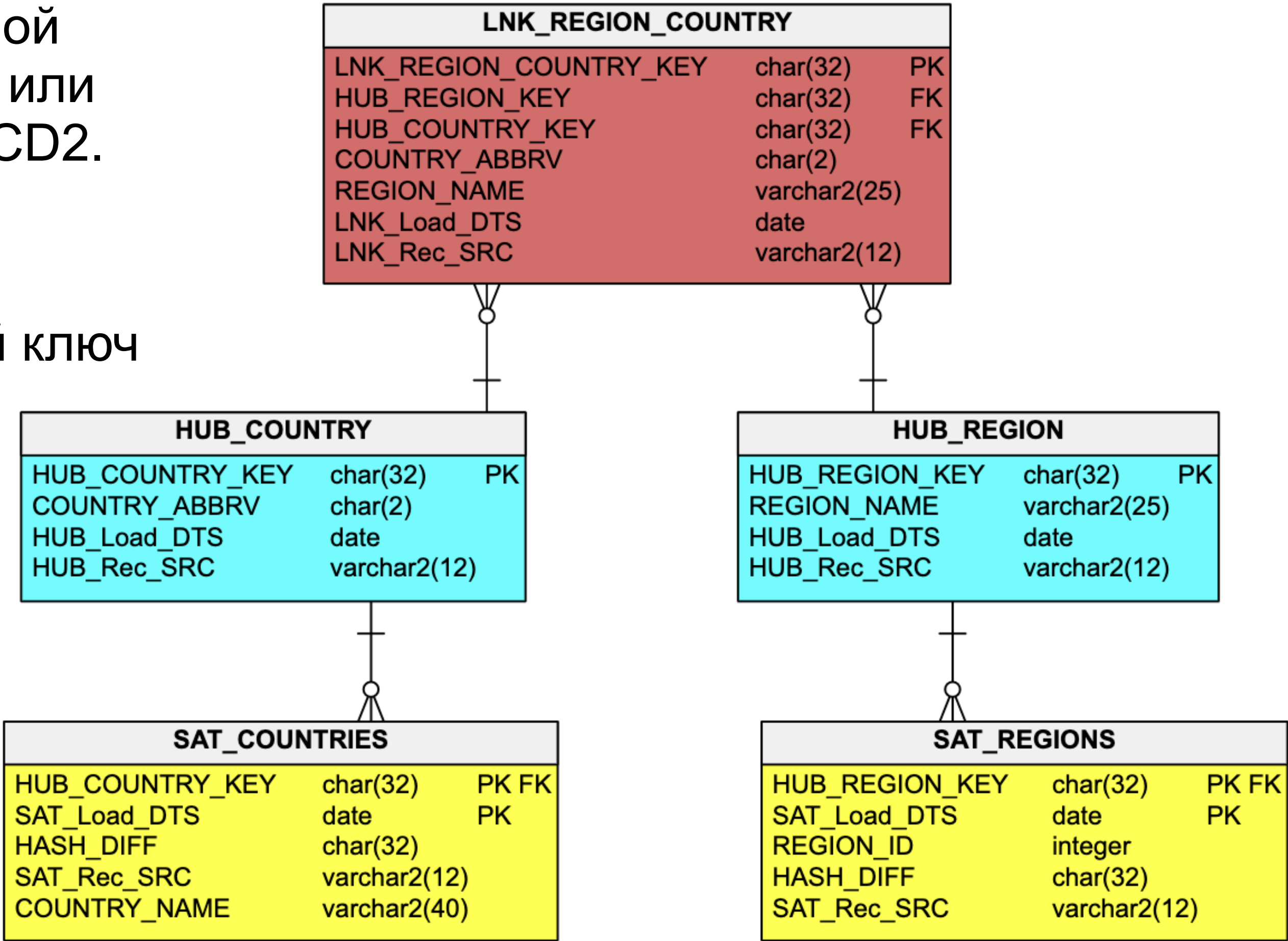


Satellite

Сателлиты (Satellite) являются контекстной (описательной) информацией ключа Хаба или Линка (Связи), обычно с историзмом по SCD2.

Атрибуты саттелита включают:

- › Первичный ключ Сателлита: Первичный ключ Хаба или первичный ключ Связи
- › Даты действия записи (SCD2)
- › Временная отметка даты загрузки
- › Код источника данных



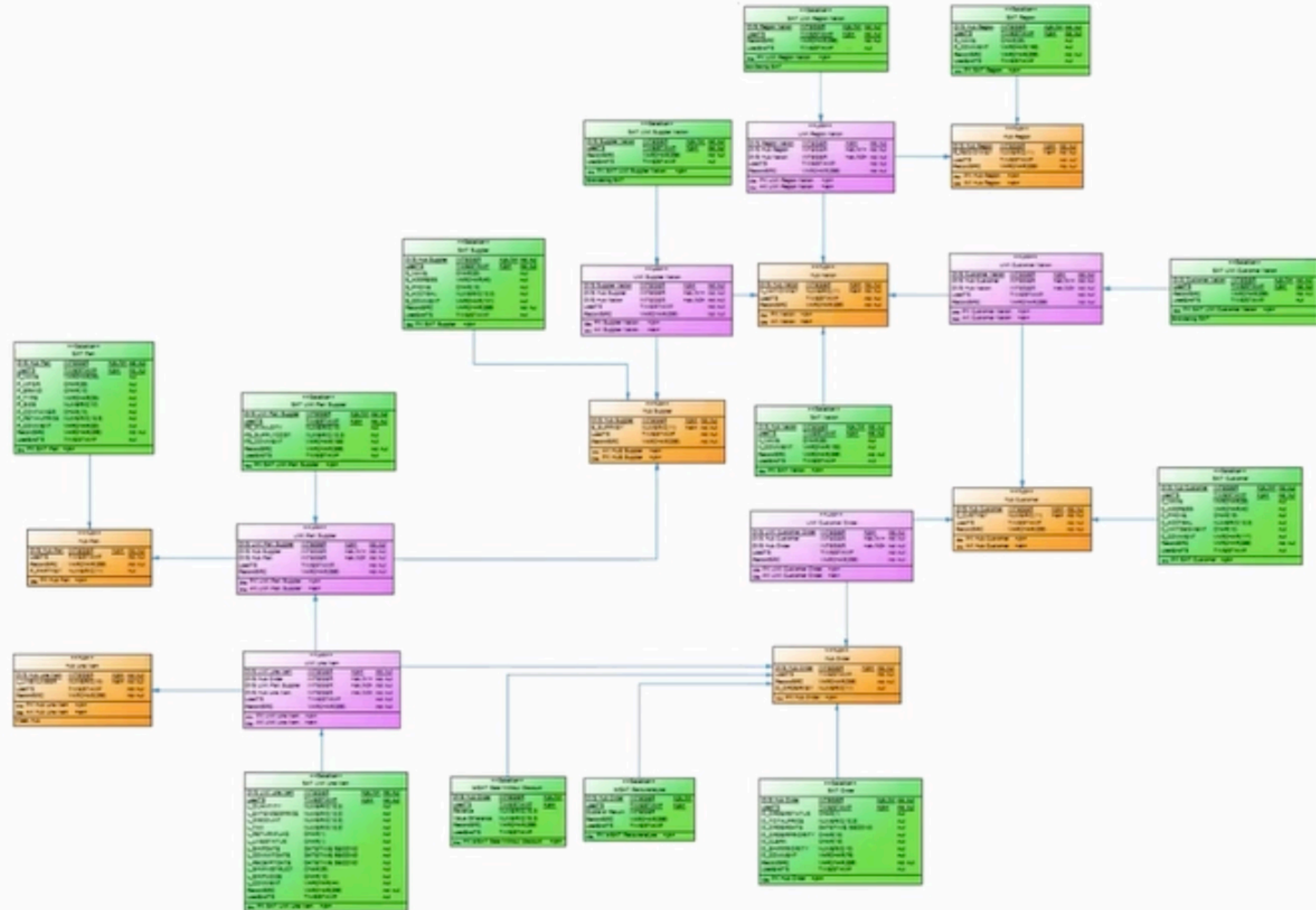
Data Vault

Data vault 1.0:

- › Хаб (Hub),
- › Связь (Link)
- › Сателлит (Satellite)

Data Vault 2.0:

- › Peg-legged link
- › Pit – Point in time
- › Bridge



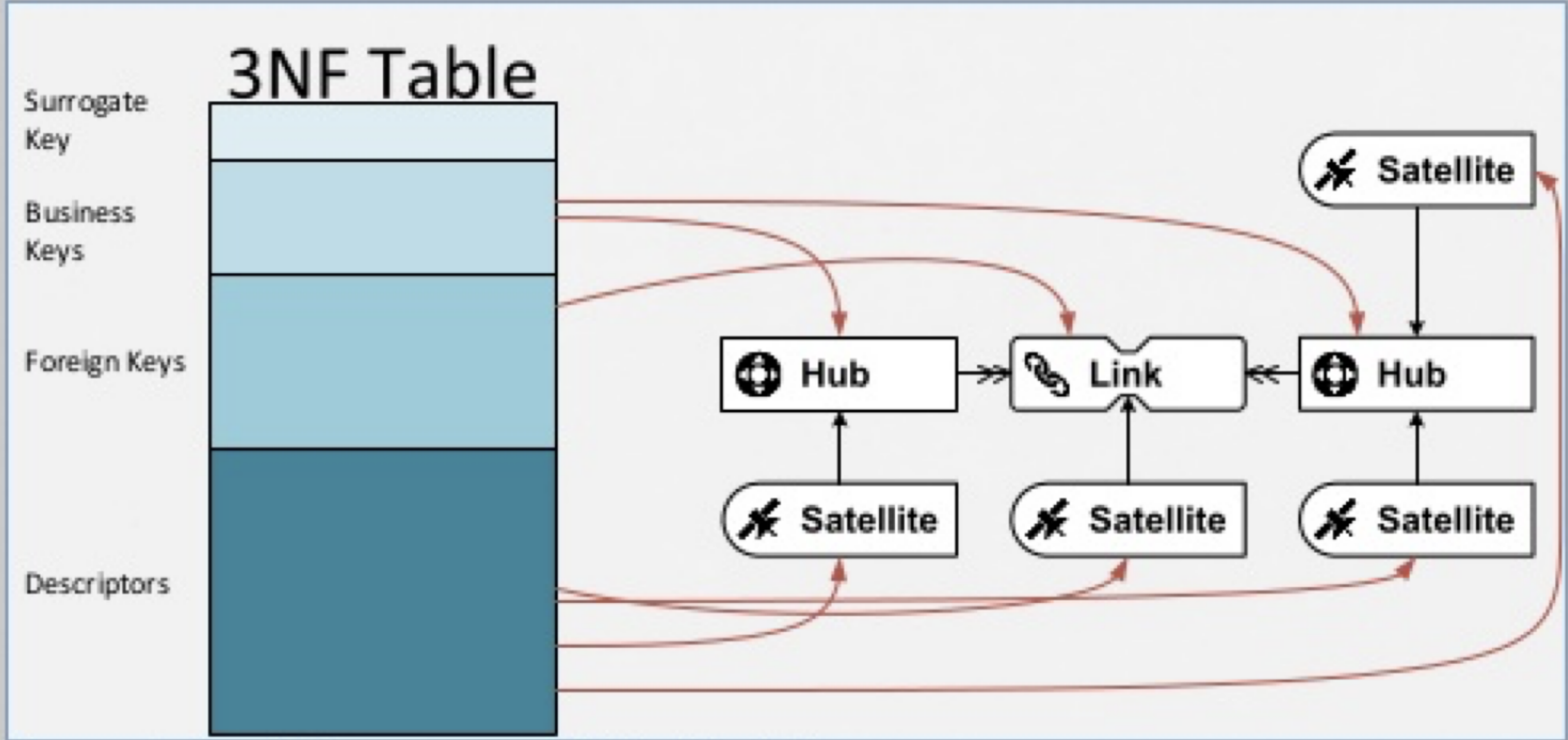
Проектирование Data Vault

Загрузка происходит следующим образом:

- › Из источников данные собираются в **стейджинг**
- › Стейджинг прогружается в **хабы**
- › В хабах генерируются **суррогатные ключи**
- › Имея суррогатные ключи, можно генерировать **сателлиты и линки**

Проектирование Data Vault

DATA VAULT 2.0 MODELING



In accordance with its own representation Linstead, 2014

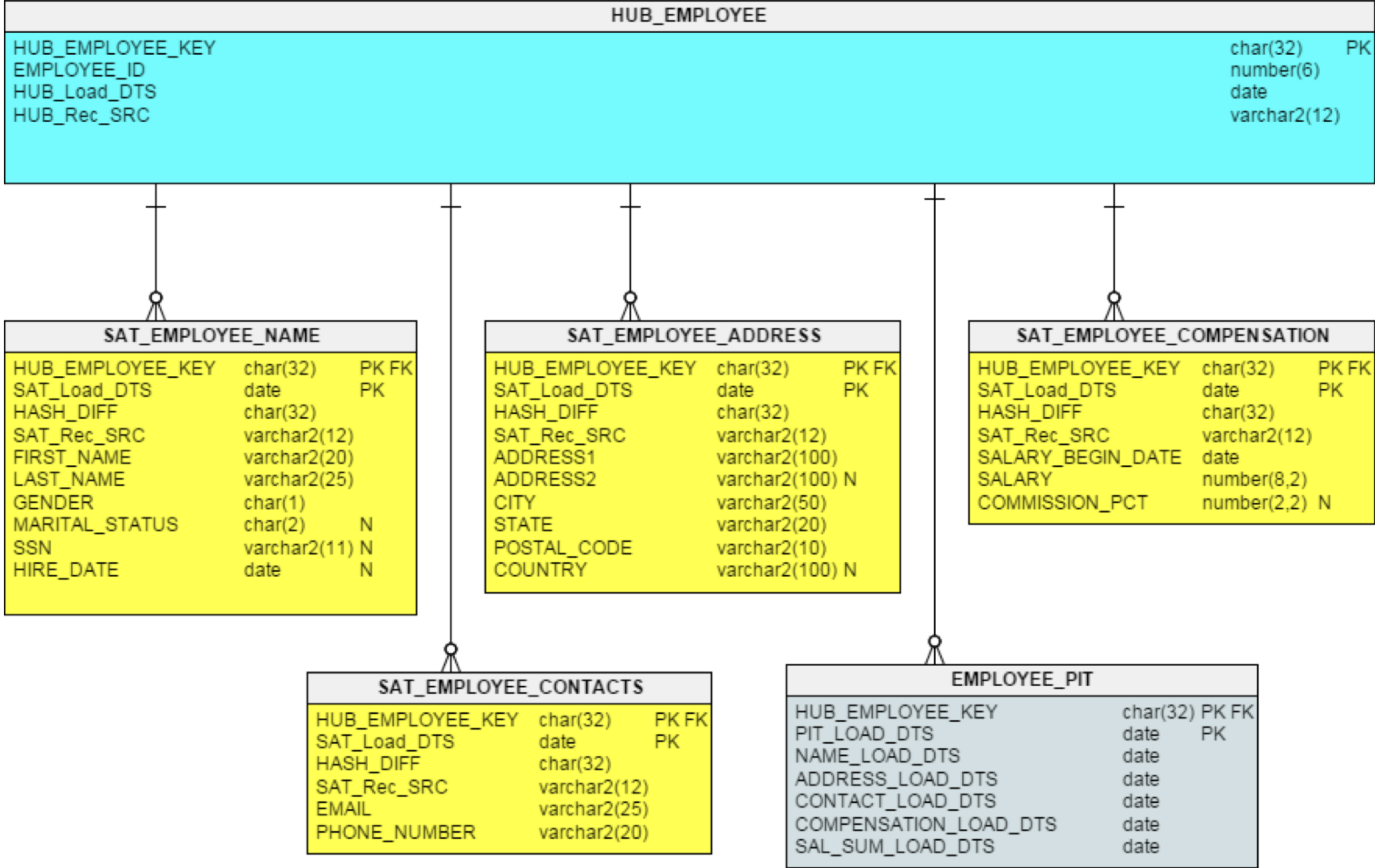


DataVault 2.0 - PiT

› **PIT (point in time)** упрощает получение информации из Сателлитов одной сущности с разной частотой обновления.

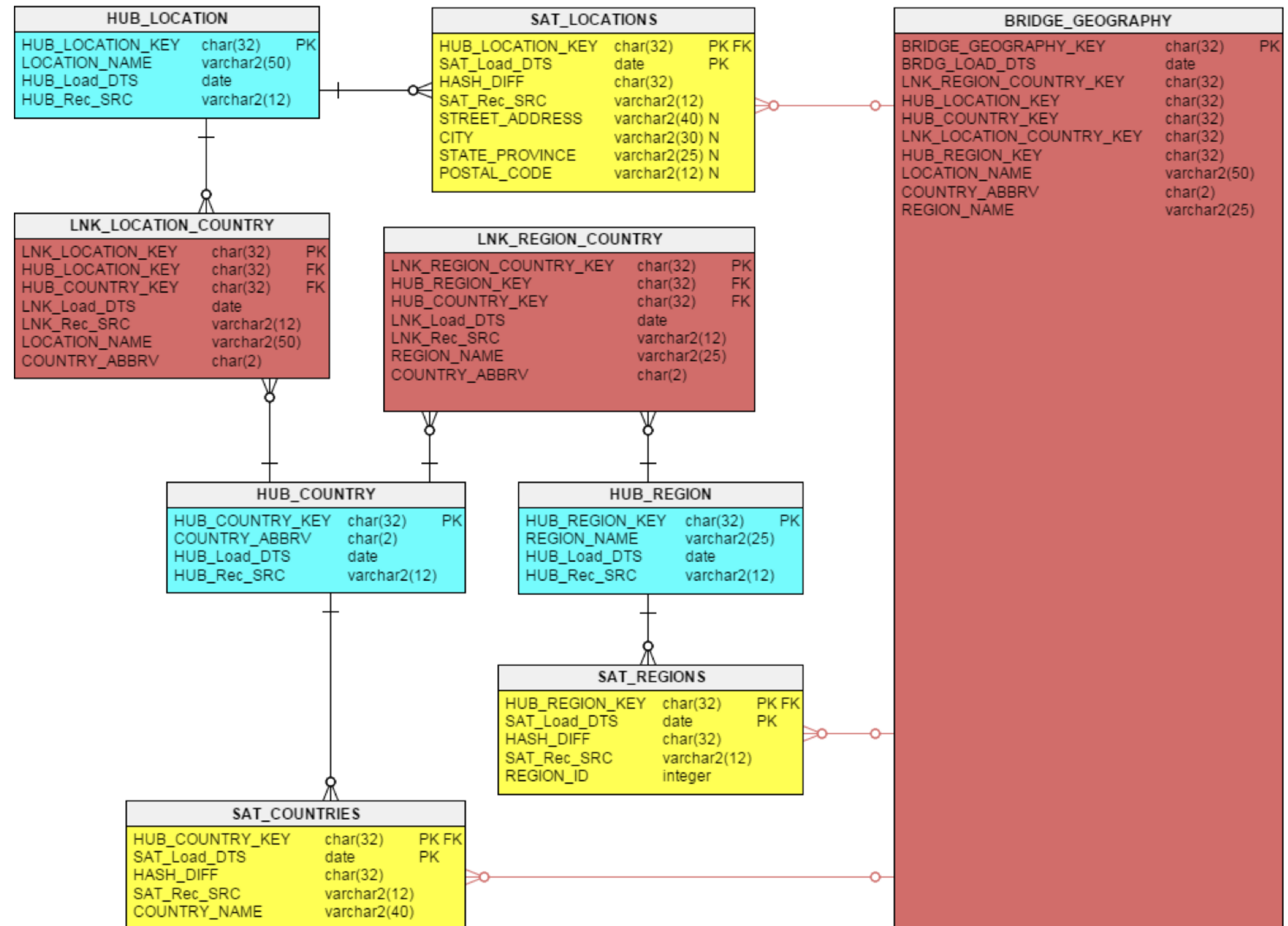
› Так как частота изменений каждого отдельного Сателлита может быть разной, не все диапазоны SCD2 будут пересекаться. В связи с этим могут возникнуть трудности с восстановлением состояния записи на некоторый момент времени.

› В PIT-таблице добавляются записи на каждое изменение каждого Сателлита. Фактически, это позволяет с помощью join'ов Хаба на PIT и Сателлиты воссоздать всю сущность на необходимый момент времени.



DataVault 2.0 - Bridge

- › **Мост (Bridge)** упрощает соединение данных через несколько связей.
- › При наличии нескольких Линков, соединение всех таблиц в некоторую сущность может быть затруднено. Bridge упрощает это, так как фактически содержит в себе все Линки.

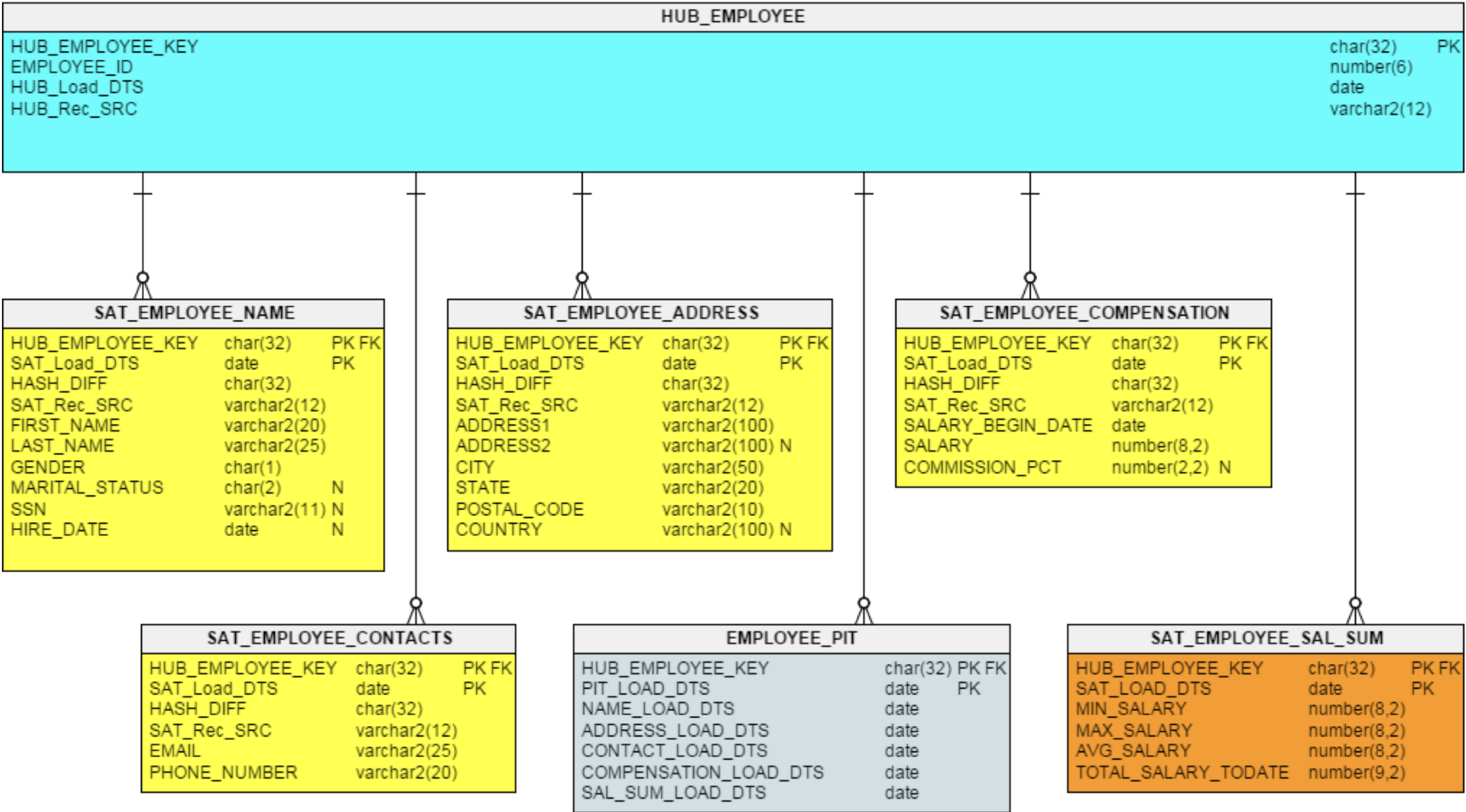


DataVault 2.0 - Predefined Derivations

› Predefined Derivations

(предагрегаты) представляет собой готовые, предагрегированные (или предкалькулированные) данные, такие как дата первой поездки или минимальное время нахождения на сервисе.

› Эти данные появляются на следующем этапе после загрузки Сателлитов.



Модно?



4 - Anchor Modeling

Anchor Modeling

Автор: Ларс Рёнбэк (Lars Rönnbäck)

Якорное моделирование - это технология моделирования гибкой базы данных, подходящая для информации, которая со временем изменяется как по структуре, так и по содержанию.

В методике моделирования используются четыре модели моделирования: **якорь атрибут, связь и узел**, каждый из которых отражает различные аспекты моделируемого домена. Полученные модели могут быть переведены в физические проекты баз данных с использованием формализованных правил. Когда такой перевод сделан, таблицы в реляционной базе данных будут в основном в шестой нормальной форме.

Anchor Modeling

Anchor (Якорь) — это существительное, объект реального мира.

Anchor таблица должна хранить только суррогатный ключ и несколько технических полей (система-источник, дата-время загрузки).

Пример - товар, пользователь, платеж

Attribute (Атрибут) — это таблица для хранения свойства, атрибута объекта

Пример - название товара, логин и дата рождения пользователя, суммы платежа.

Одно свойство у объекта — одна Attribute-таблица.

Каждая Attribute-таблица содержит суррогатный ключ объекта, которым является ссылка на соответствующий Anchor, поле для значения атрибута, и, опционально, дату для историчности и технические поля.

Anchor Modeling

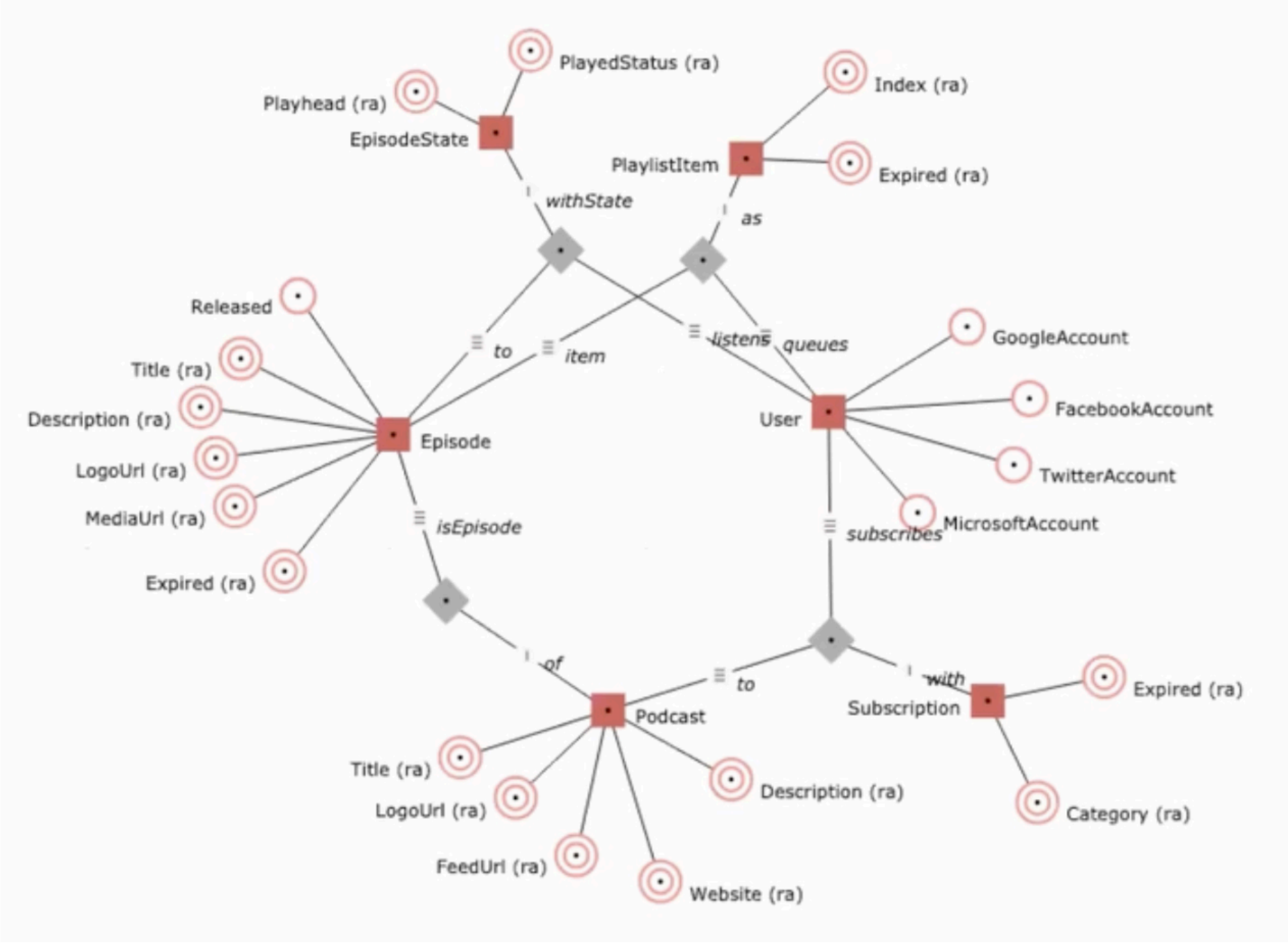
Tie (Связь) — это таблица для хранения связей между объектами.

Таблица должна содержать суррогатный ключ левого объекта (customer_id), правого объекта (country_id) и, по необходимости, даты историчности и технических полей.

Knot (Узел) - таблица-состояние, справочник. Узлы можно рассматривать как сочетание якоря и одного атрибута.

Пример – справочник, в котором содержатся поля.

Как это выглядит?



Проектирование

- › Найдите основные бизнес-сущности (это якоря)
- › Исследуйте их (найдите другие якоря, связи)
- › Опишите их (атрибуты, узлы)
- › Историзируйте их, если нужно (атрибуты)
- › Определите взаимоотношения сущностей (связи)
- › Историзируйте взаимоотношения (связи)

Технические нюансы реализации Anchor modeling

- › Якорная модель крайне чувствительна к техническим возможностям СУБД
- › ОГРОМНОЕ количество join
- › Table (join) elimination
- › Cluster index
- › Insert Only (обычно, но не всегда)
- › Необходима автоматизация

Что выбрать?

сложность эксплуатации, простота внесения изменений

Никакого

- › Денормализация
- › Можно использовать без подготовки
- › Неустойчиво к изменениям
- › Дублирование информации
- › Нет join

Звезда и снежинка

- › Нормализация
- › Можно использовать с минимальной подготовкой
- › Неудобно перестраивать
- › Минимальное дублирование информации
- › Приемлемое количество join

Data Vault

- › Строгая нормализация
- › Нельзя использовать без подготовки
- › Не надо перестраивать
- › Нет дублирования информации
- › Большое количество join

Anchor modeling

- › Ультра нормализация
- › Нельзя использовать без подготовки
- › Не надо перестраивать
- › Нет дублирования информации
- › Ультра количество join

легкость эксплуатации, сложность внесения изменений

Дилемма проектирования DWH



Дилемма проектирования DWH

DM



5 - Домашняя работа #2

6 - ETL

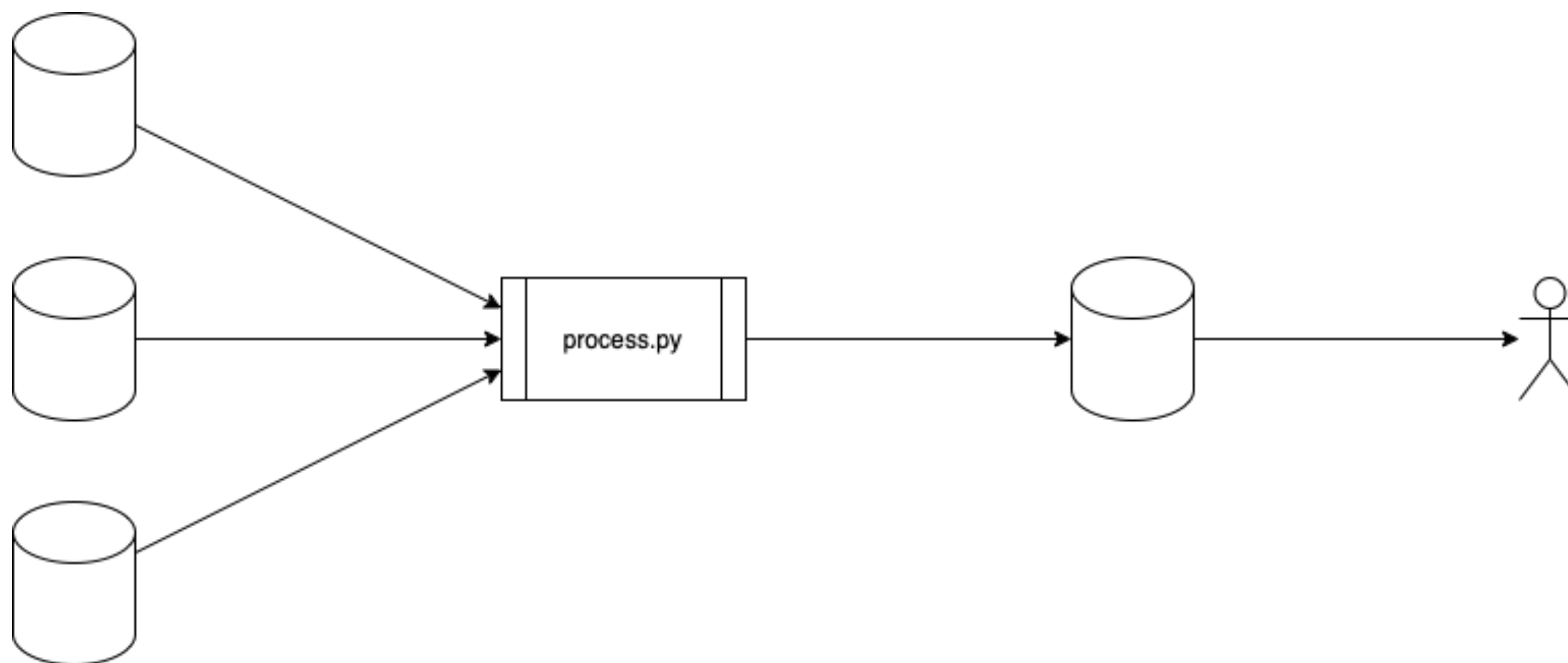
Что такое ETL

ETL (Extract, Transform, Load) - это класс процессов управления данными. Он состоит из:

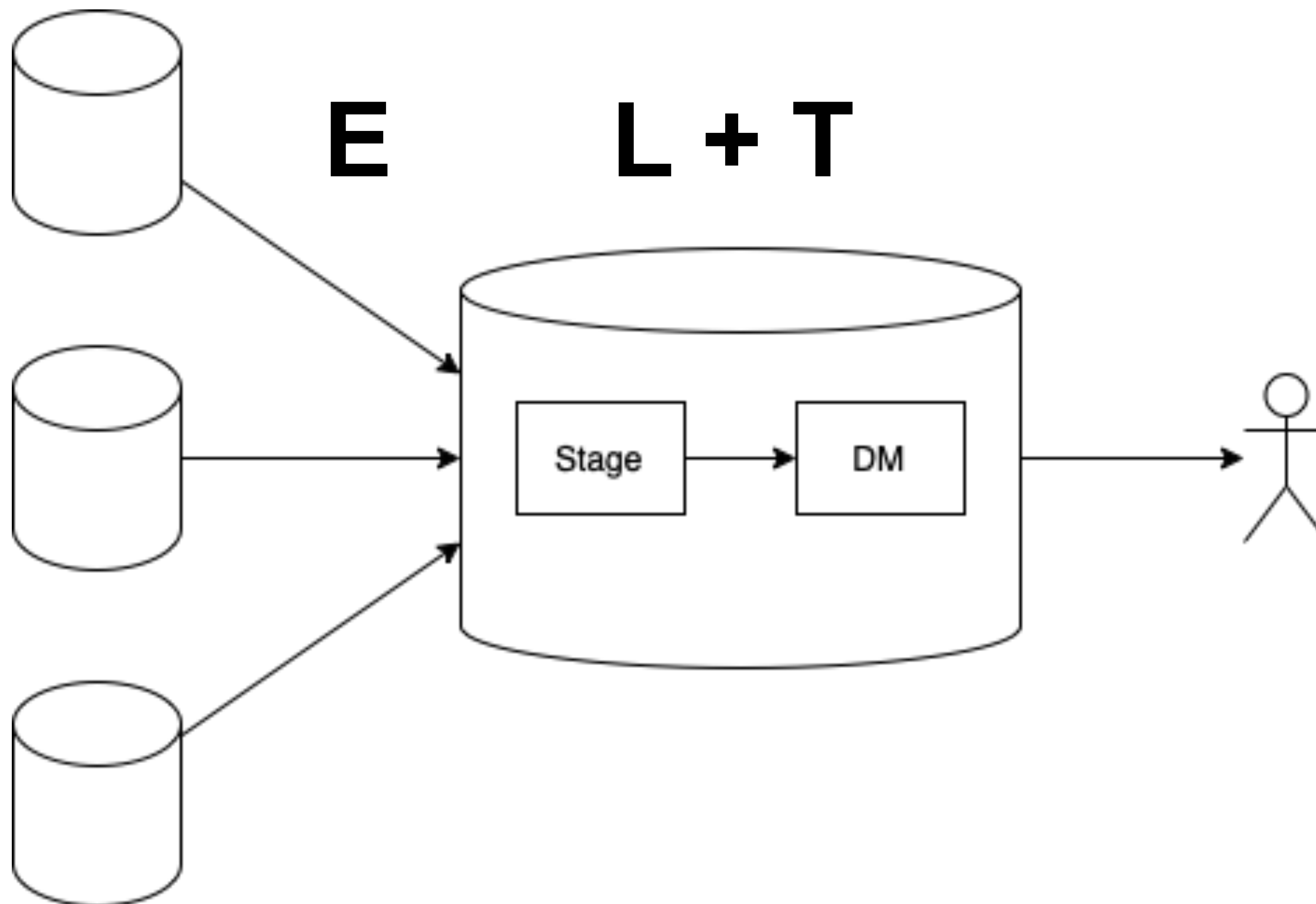
- › **Извлечение данных (Extraction - E)** - из одного или нескольких источников и подготовка их к преобразованию (загрузка в промежуточную область, проверка данных на соответствие спецификациям и возможность последующей загрузки в ХД);
- › **Трансформация данных (Transform - T)** - преобразование форматов и кодировки, агрегация и очистка;
- › **Загрузка данных (Load - L)** - запись преобразованных данных, включая информацию о структуре их представления (метаданные) в необходимую систему хранения или витрину данных.

Какие бывают ETL

E T L



Какие бывают ETL



Свойства ETL-процесса

Мы с вами будем говорить про 3 свойства разных видов ETL:

- › Идемпотентность
- › Delete Policy
- › Change Tracking

Идемпотентность

Идемпотентность - гарантия того, что при повторном применении трансформации к данным результат будет таким же, что и при первом.

Это свойство может гарантироваться только в случае, когда на вход подаются одинаковые данные.

Возможные значения:

- › True
- › False

Delete Policy

Delete Policy определяет способ обработки факта удаления данных на источнике при трансформации.

Возможные значения:

- › **Never** - данные из целевой таблицы не могут быть удалены в штатном режиме работы даже при их удалении с источника
- › **Delete** - удаление данных на источнике приводит к удалению данных в целевой таблице
- › **Delete on backfill** - удаление данных может произойти только при историческом пересчете

Change Tracking

Change Tracking определяет, выполняется ли построение истории изменения объекта по его идентификатору или нет. Существуют различные способы сохранения этой истории.

Возможные значения:

- › No tracking
- › Overwrite
- › History
- › Accumulate

Snapshot

Полная перезапись таблицы из источников.

Snapshot

Данные в целевой таблице

id=1
id=2
id=3
id=4

Новые данные

id=1
id=3
id=4
id=5

Результат

id=1
id=3
id=4
id=5

Snapshot

Требования к целевой таблице:

- › Никаких

Когда можно:

- › Маленькие таблицы

- › Нужно хранить только текущее состояние

Когда нельзя:

- › Большие таблицы (1M+ rows)

Snapshot

Требования к целевой таблице:

- › Никаких

Когда можно:

- › Маленькие таблицы
- › Нужно хранить только текущее состояние

Когда нельзя:

- › Большие таблицы (1M+ rows)

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

Snapshot

Требования к целевой таблице:

- › Никаких

Когда можно:

- › Маленькие таблицы
- › Нужно хранить только текущее состояние

Когда нельзя:

- › Большие таблицы (1M+ rows)

Вопросы:

- › Идемпотентность
True
- › Delete Policy
Delete
- › Change Tracking
Overwrite

Periodic Snapshot

Загружаем обновления за последний период с некоторой
гранулярностью: час, день, неделя, месяц, квартал, год и т.д.

Periodic Snapshot за 2020-01-02								
Данные в целевой таблице			Новые данные			Результат		
dt=2020-01-01	id=1	v=10	dt=2020-01-02	id=1	v=15	dt=2020-01-01	id=1	v=10
dt=2020-01-01	id=2	v=20	dt=2020-01-02	id=3	v=30	dt=2020-01-01	id=2	v=20
dt=2020-01-01	id=3	v=30	dt=2020-01-02	id=4	v=40	dt=2020-01-01	id=3	v=30
						dt=2020-01-02	id=1	v=15
						dt=2020-01-02	id=3	v=30
						dt=2020-01-02	id=4	v=40

Periodic Snapshot

Требования к целевой таблице:

- › Должно быть поле с датой загрузки
- › Может быть партицирована

Когда можно:

- › Мало данных
- › Надо знать состояние сущности с какой-то гранулярностью

Когда нельзя:

- › Большие таблицы, т.к. избыточность

Periodic Snapshot

Требования к целевой таблице:

- › Должно быть поле с датой загрузки
- › Может быть партицирована

Когда можно:

- › Мало данных
- › Надо знать состояние сущности с какой-то гранулярностью

Когда нельзя:

- › Большие таблицы, т.к. избыточность

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

Periodic Snapshot

Требования к целевой таблице:

- › Должно быть поле с датой загрузки
- › Может быть партицирована

Когда можно:

- › Мало данных
- › Надо знать состояние сущности с какой-то гранулярностью

Когда нельзя:

- › Большие таблицы, т.к. избыточность

Вопросы:

- › Идемпотентность
True
- › Delete Policy
Delete on backfill
- › Change Tracking
Accumulate

Append

Старые данные не меняются. Новые данные дозаписываются.

Append

Данные в целевой таблице

dt=2020-01-01	id=1	v=10
dt=2020-01-01	id=2	v=20
dt=2020-01-01	id=3	v=30

Новые данные

dt=2020-01-02	id=1	v=15
dt=2020-01-02	id=3	v=30
dt=2020-01-02	id=4	v=40

Результат

dt=2020-01-01	id=1	v=10
dt=2020-01-01	id=2	v=20
dt=2020-01-01	id=3	v=30
dt=2020-01-02	id=1	v=15
dt=2020-01-02	id=3	v=30
dt=2020-01-02	id=4	v=40

Append

Требования к целевой таблице:

- › Никаких

Когда можно:

- › Данные не меняются после записи
- › Для каких-то целей нужен лог изменений

Когда нельзя:

- › При большом кол-ве изменений таблица будет быстро расти
- › Может потребоваться TTL

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

Append

Требования к целевой таблице:

- › Никаких

Когда можно:

- › Данные не меняются после записи
- › Для каких-то целей нужен лог изменений

Когда нельзя:

- › При большом кол-ве изменений таблица будет быстро расти
- › Может потребоваться TTL

Вопросы:

- › Идемпотентность
False
- › Delete Policy
Never
- › Change Tracking
Accumulate

Insert by key

Если появятся новые данные - они добавляются в таблицу. Старые данные не меняются. При удалении из источника данные не удаляются.

Insert

Данные в целевой таблице

id=1	value=10
id=2	value=20
id=3	value=30

Новые данные

id=1	value=10
id=2	value=25
id=4	value=40

Результат

id=1	value=10
id=2	value=20
id=3	value=30
id=4	value=40

Insert by key

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину

- › Нужно хранить только первое состояние

Когда нельзя:

- › Нет уникального ключа

Insert by key

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину
- › Нужно хранить только первое состояние

Когда нельзя:

- › Нет уникального ключа

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

Insert by key

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину
- › Нужно хранить только первое состояние

Когда нельзя:

- › Нет уникального ключа или его тяжело брать

Вопросы:

- › Идемпотентность
True
- › Delete Policy
Never
- › Change Tracking
No tracking

Upsert

Если появятся новые данные - они добавляются в таблицу. Старые данные таблицы обновляются из новых по ключу. При удалении из источника данные не удаляются.

Upsert

Данные в целевой таблице

id=1	value=10
id=2	value=20
id=3	value=30

Новые данные

id=1	value=10
id=2	value=25
id=4	value=40

Результат

id=1	value=10
id=2	value=25
id=3	value=30
id=4	value=40

Upsert

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину

- › Нужно хранить текущее состояние

Когда нельзя:

- › Нет уникального ключа

Upsert

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину
- › Нужно хранить текущее состояние

Когда нельзя:

- › Нет уникального ключа

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

Upsert

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Данные могут меняться за произвольную глубину
- › Нужно хранить текущее состояние

Когда нельзя:

- › Нет уникального ключа

Вопросы:

- › Идемпотентность
True
- › Delete Policy
Never
- › Change Tracking
Overwrite

History по SCD2

History SCD2

Данные в целевой таблице

id=1	name=a	v=10	from=2019-01-01	to=2019-12-31
id=1	name=a	v=10	from=2020-01-01	to=9999-12-31

Новые данные за 2020-02-01

id=1	name=a	v=20
id=2	name=b	v=15

Результат

id=1	name=a	v=10	from=2019-01-01	to=2019-12-31
id=1	name=a	v=10	from=2020-01-01	to=2020-01-31
id=1	name=a	v=20	from=2020-02-01	to=9999-12-31
id=2	name=b	v=15	from=2020-02-01	to=9999-12-31

Берется срез изменений за определенную дату.
На основе этого среза изменяются или создаются сроки в исходной таблице.

History по SCD2

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Нужно отслеживать историю изменения атрибутов

- › Чаще всего применяется для справочников

Когда нельзя:

- › При большом количестве изменений таблица быстро растёт в размере

History по SCD2

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Нужно отслеживать историю изменения атрибутов
- › Чаще всего применяется для справочников

Когда нельзя:

- › При большом количестве изменений таблица быстро растёт в размере

Вопросы:

- › Идемпотентность?
- › Delete Policy?
- › Change Tracking?

History по SCD2

Требования к целевой таблице:

- › Нужен уникальный ключ

Когда можно:

- › Нужно отслеживать историю изменения атрибутов
- › Чаще всего применяется для справочников

Когда нельзя:

- › При большом количестве изменений таблица быстро растёт в размере

Вопросы:

- › Идемпотентность
True
- › Delete Policy
Never
- › Change Tracking
History

Особенности времени

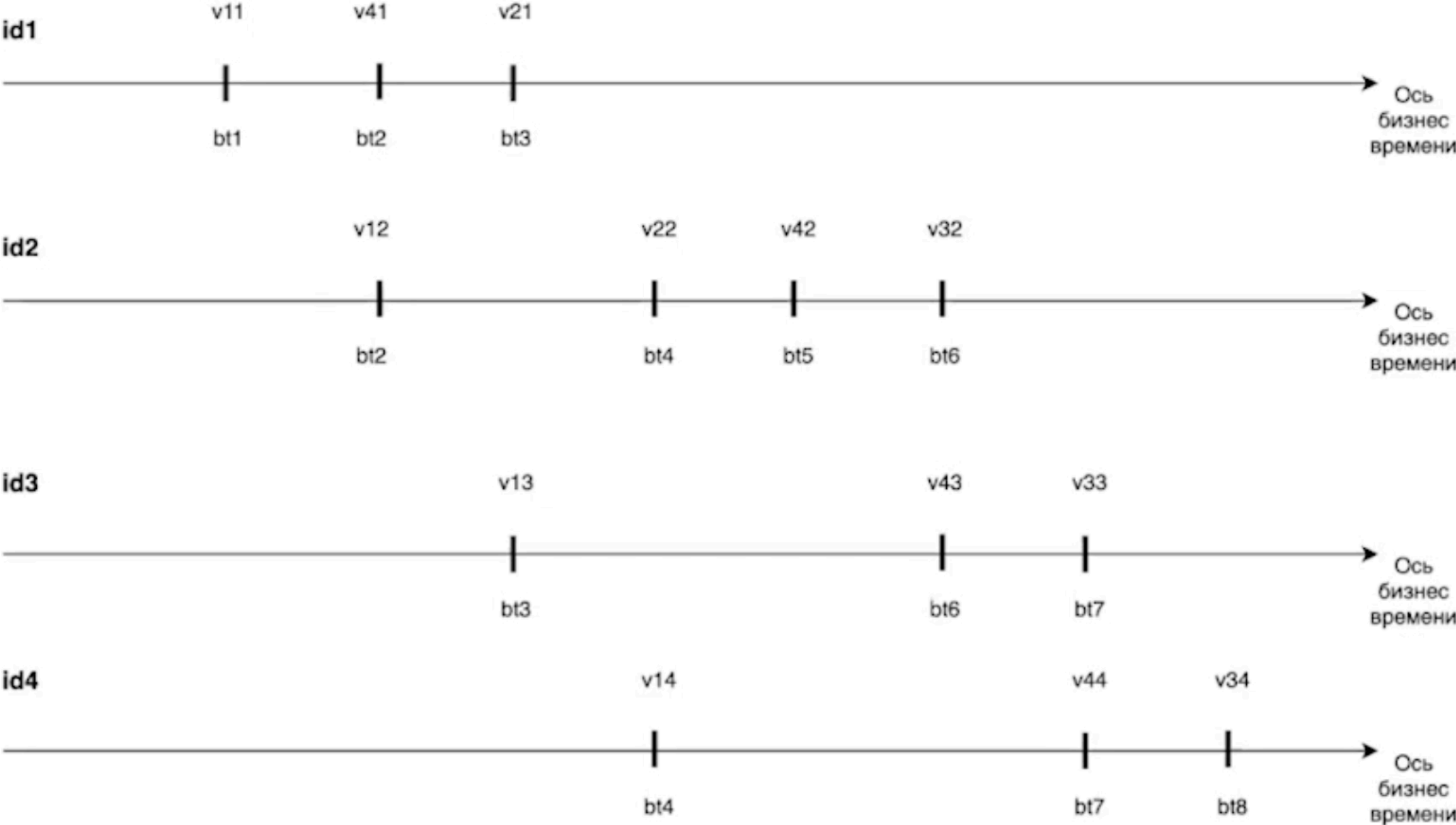
У нас есть 2 вида времени:

- › **Бизнес-время** - время актуальности данных с точки зрения бизнеса; при получении данных с бизнес-временем мы можем говорить, что они актуальны на указанное бизнес-время;
- › **Техническое время** - реальное время на сервере при получении партии данных;

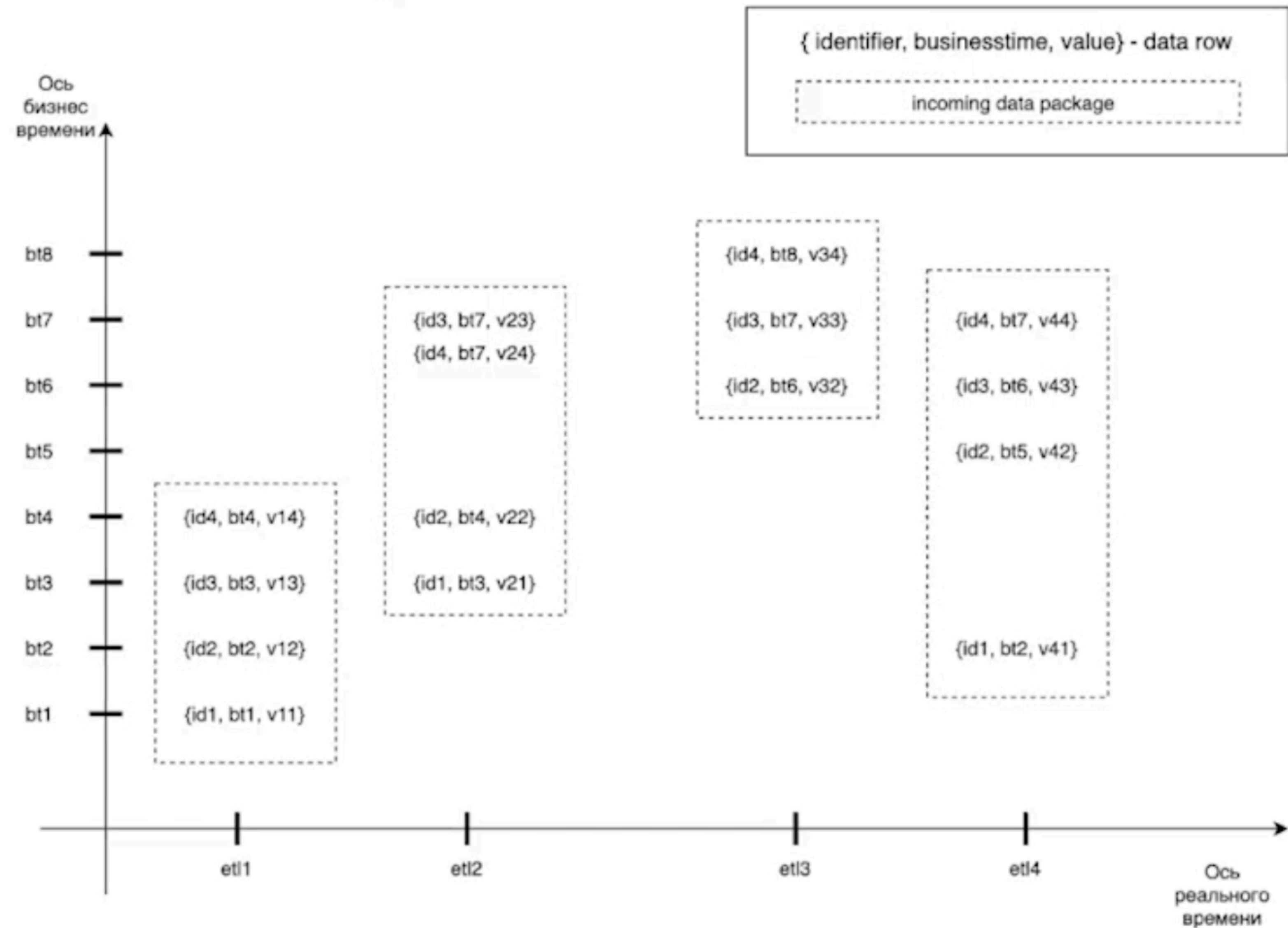
Свойства времен:

- › Техническое время монотонно возрастает - мы не можем получать данные в прошлом
- › Бизнес-время может приходить за любой период - прошлое, настоящее или будущее
- › Для одной бизнес-сущности не может быть двух значений данных в одно бизнес-время

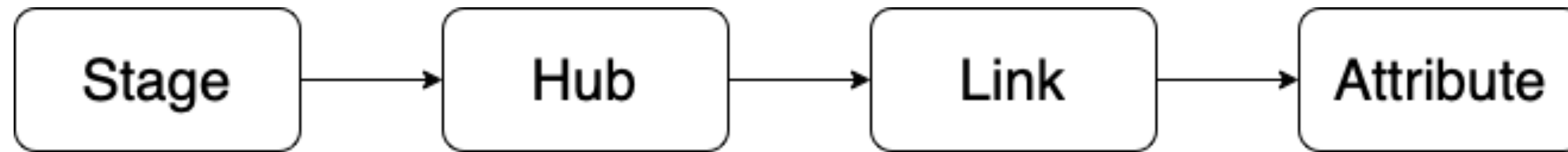
Бизнес-время



Техническое время



Как работает загрузка в DWH



- › Грузим данные в STG
- › Генерируем суррогатные ключи для Hub
- › Грузим Hub
- › Грузим Link
- › Грузим Attribute

Как работает загрузка в DWH

